

# Relatório de Nivelamento: Indução Completa, Expansão Telescópica e F.O.C.A.

Docente: Prof. Dr. Diego Gervasio Frias Suarez  
Discente: Guilherme Dos Santos Modesto Teixeira

## Introdução

A análise de algoritmos iterativos e recursivos exige, frequentemente, a contagem exata do número de execuções de determinadas operações. Para laços iterativos aninhados, essa contagem se traduz em somatórios ou séries matemáticas. Já para funções recursivas, o número de repetições é descrito por relações de recorrência. Este relatório tem como objetivo apresentar, de forma concisa, dois métodos matemáticos fundamentais para a resolução desses problemas: o método da **Indução Completa** (ou Matemática) e o método da **Expansão Telescópica**. Além disso, será apresentado o paradigma **F.O.C.A.** (Funções, Operações Aritméticas, Comparações e Atribuições), uma ferramenta prática para a contagem detalhada de operações primitivas em trechos de código.

## 1 Indução Completa

O método da Indução Completa é uma técnica de demonstração matemática utilizada para provar que uma determinada propriedade  $P(n)$  é verdadeira para todos os números naturais  $n$  a partir de um valor inicial  $n_0$ . A lógica é análoga ao efeito dominó: se o primeiro dominó cai (passo base) e se cada dominó, ao cair, derruba o próximo (passo indutivo), então todos os dominós cairão.

O método é composto por dois passos fundamentais:

1. **Passo Base:** Verifica-se que a propriedade  $P(n)$  é verdadeira para o menor valor do domínio, geralmente  $n = 1$  ou  $n = 0$ .
2. **Passo Indutivo:** Assume-se que a propriedade é verdadeira para um valor  $n = k$  (hipótese de indução) e, a partir dessa suposição, demonstra-se que a propriedade também é verdadeira para  $n = k + 1$ .

Se ambos os passos forem satisfeitos, conclui-se que  $P(n)$  é válida para todo  $n \geq n_0$ .

### Exemplo Prático: Soma dos $n$ primeiros números naturais

**Proposição:** A soma dos  $n$  primeiros números naturais é dada por:

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

#### Demonstração por Indução:

Seja  $P(n)$  a afirmação de que  $\sum_{i=1}^n i = \frac{n(n+1)}{2}$ .

1. **Passo Base:** Para  $n = 1$ , temos:

$$\sum_{i=1}^1 i = 1 \quad \text{e} \quad \frac{1(1+1)}{2} = \frac{2}{2} = 1$$

Portanto,  $P(1)$  é verdadeira.

2. **Passo Indutivo:** Suponha que  $P(k)$  seja verdadeira para algum  $k \geq 1$ , ou seja:

$$1 + 2 + \dots + k = \frac{k(k+1)}{2}$$

Precisamos provar que  $P(k+1)$  também é verdadeira. Para isso, partimos da soma até  $k+1$ :

$$1 + 2 + \dots + k + (k+1) = (1 + 2 + \dots + k) + (k+1)$$

Usando a hipótese de indução, substituímos a soma dos  $k$  primeiros termos:

$$\begin{aligned} &= \frac{k(k+1)}{2} + (k+1) = (k+1) \left( \frac{k}{2} + 1 \right) = (k+1) \left( \frac{k+2}{2} \right) \\ &= \frac{(k+1)(k+2)}{2} \end{aligned}$$

Que é exatamente a fórmula para  $n = k+1$ . Portanto,  $P(k+1)$  é verdadeira.

Como o passo base e o passo indutivo foram verificados, concluímos que a fórmula  $\sum_{i=1}^n i = \frac{n(n+1)}{2}$  é válida para todo  $n \in \mathbb{N}$ .

## 2 Expansão Telescópica

A Expansão Telescópica é um método para resolver relações de recorrência lineares de primeira ordem, especialmente aquelas da forma  $T(n) = T(n-1) + f(n)$ . A técnica consiste em escrever a recorrência para valores sucessivos de  $n$ , de modo que, ao somar todas as equações, os termos  $T(n-1)$ ,  $T(n-2)$ , etc., se cancelem, restando apenas o termo  $T(n)$  e a soma dos  $f(i)$ .

### Exemplo Prático: Número de iterações de um laço aninhado dependente

Considere o seguinte código, que conta o número de iterações de um laço aninhado onde o limite do laço interno depende do índice do laço externo:

```
iteracoes = 0
for i in range(1, n + 1):
    for j in range(1, i + 1):
        iteracoes += 1
```

O número total de iterações é dado pela soma  $1 + 2 + \dots + n$ . Este valor pode ser descrito por uma relação de recorrência. Seja  $T(n)$  o número total de iterações para um dado  $n$ . Observando o comportamento da função, podemos definir a recorrência:

$$T(n) = T(n-1) + n$$

com a condição inicial  $T(1) = 1$ . O termo  $n$  representa o número de iterações do laço interno quando o laço externo está na iteração  $i = n$ .

### Resolução por Expansão Telescópica:

Escrevemos a recorrência para os valores de  $n$  até a condição inicial:

$$\begin{aligned} T(n) &= T(n-1) + n \\ T(n-1) &= T(n-2) + (n-1) \\ &\vdots \\ T(2) &= T(1) + 2 \end{aligned}$$

Para facilitar o cancelamento, reescrevemos cada equação isolando a diferença:

$$\begin{aligned} T(n) - T(n-1) &= n \\ T(n-1) - T(n-2) &= n-1 \\ &\vdots \\ T(2) - T(1) &= 2 \end{aligned}$$

Somamos todas as equações. No lado esquerdo, todos os termos intermediários se cancelam (telescopia), restando apenas  $T(n) - T(1)$ :

$$T(n) - T(1) = 2 + 3 + \dots + n$$

Isolando  $T(n)$  e substituindo  $T(1) = 1$ :

$$T(n) = 1 + 2 + 3 + \dots + n$$

Obtemos, portanto, a mesma série que já conhecemos. Usando a fórmula provada por indução na seção anterior:

$$T(n) = \frac{n(n+1)}{2}$$

Este exemplo mostra como a expansão telescópica transforma uma relação de recorrência em um somatório, que pode então ser resolvido por métodos conhecidos (como a indução).

### 3 O Paradigma F.O.C.A. (Funções, Operações Aritméticas, Comparações e Atribuições)

O paradigma F.O.C.A. é uma metodologia sistemática para a contagem do número exato de operações primitivas executadas por um trecho de código. Ele descreve a forma como devemos analisar cada linha de um algoritmo, decompondo-a em quatro componentes principais:

- **Funções (F):** Número de chamadas a funções (incluindo funções internas, como `range`, `len`, ou funções definidas pelo usuário).
- **Operações Aritméticas (O):** Contagem de operações como soma, subtração, multiplicação, divisão, incremento, decremento, etc.
- **Comparações (C):** Número de testes relacionais realizados, incluindo as condições de laços (`for`, `while`) e estruturas condicionais (`if`, `elif`).
- **Atribuições (A):** Número de vezes que uma variável recebe um valor (incluindo inicializações e atualizações).

A aplicação do F.O.C.A. permite obter uma função de custo detalhada, que pode ser usada para comparar a eficiência de diferentes implementações e para inferir a complexidade assintótica do algoritmo.

#### Exemplo Prático: Custo do Cálculo de um Elemento na Multiplicação de Matrizes

Considere o trecho de código que calcula um único elemento  $c[i, j]$  da matriz produto  $C = A \times B$ , onde  $A$  tem dimensões  $n \times m$  e  $B$  tem dimensões  $m \times p$ .

```
c[i, j] = 0.0
for k in range(m):
    c[i, j] = c[i, j] + a[i, k] * b[k, j]
```

Aplicando a análise F.O.C.A.:

1. **Inicialização:** `c[i, j] = 0.0`
  - Atribuições (A): 1 (atribuição explícita).
  - Funções (F): 0.
  - Operações (O): 0.
  - Comparações (C): 0.
2. **Gestão do laço for:** `for k in range(m)`
  - O laço executa  $m$  iterações.
  - Inicialização de `k`: `A = 1` (atribuição).

- Teste de condição (executado  $m + 1$  vezes):  $C = m + 1$ .
- Incremento de  $k$  (executado  $m$  vezes):  $O = m$  (operação de adição).
- Chamada a **range**:  $F = 1$  (considerando a criação do iterador).

### 3. Corpo do laço: $c[i,j] = c[i,j] + a[i,k] * b[k,j]$

- Frequência:  $m$ .
- Operações aritméticas: 2 por iteração (uma multiplicação e uma adição)  $\rightarrow O = 2m$ .
- Atribuições: 1 por iteração (atribuição do resultado a  $c[i,j]$ )  $\rightarrow A = m$ .
- Comparações: 0.
- Funções: 0 (acessos a arrays não são contados como chamadas de função).

Somando os custos para o cálculo de um único elemento:

$$F_{\text{elemento}} = 1 \quad (\text{chamada a } \textit{range})$$

$$O_{\text{elemento}} = m \text{ (incremento)} + 2m \text{ (corpo)} = 3m$$

$$C_{\text{elemento}} = m + 1$$

$$A_{\text{elemento}} = 1 \text{ (inicialização de } c) + 1 \text{ (inicialização de } k) + m \text{ (atribuições no corpo)} = m + 2$$

Portanto, o custo total para um elemento é:

$$C_{\text{elemento}} = F + O + C + A = 1 + 3m + (m + 1) + (m + 2) = 5m + 4$$

## Fórmula Final do Número de Operações para a Multiplicação de Matrizes

Com o valor de  $C_{\text{elemento}}$  determinado, podemos agora fechar a fórmula completa do custo computacional do algoritmo. Retomando a estrutura geral do programa:

```
for i in range(n):           # loop externo
    for j in range(p):       # loop interno
        calculo de C[i,j]    # Celemento
```

O custo total é a soma dos custos de cada laço e do corpo mais interno. Vamos contabilizar separadamente cada categoria.

- **Laço externo** ( $n$  iterações):
  - $F$ : 1 (chamada a **range**)
  - $O$ :  $n$  (incrementos de  $i$ )
  - $C$ :  $n + 1$  (testes)
  - $A$ : 1 (inicialização de  $i$ )
- **Laço interno** ( $p$  iterações por iteração externa):
  - Por iteração externa:  $F = 1$ ,  $O = p$ ,  $C = p + 1$ ,  $A = 1$ .
  - Multiplicado por  $n$ :  $F = n$ ,  $O = np$ ,  $C = n(p + 1)$ ,  $A = n$ .
- **Corpo** (cálculo do elemento):
  - Executado  $n \cdot p$  vezes.
  - Cada execução custa  $F = 1$ ,  $O = 3m$ ,  $C = m + 1$ ,  $A = m + 2$ .
  - Total:  $F = np$ ,  $O = 3npm$ ,  $C = np(m + 1)$ ,  $A = np(m + 2)$ .

Somando todas as contribuições:

$$\begin{aligned}
F_{\text{total}} &= 1 \text{ (externo)} + n \text{ (interno)} + np \text{ (corpo)} = 1 + n + np \\
O_{\text{total}} &= n \text{ (externo)} + np \text{ (interno)} + 3npm \text{ (corpo)} = n + np + 3npm \\
C_{\text{total}} &= (n + 1) \text{ (externo)} + n(p + 1) \text{ (interno)} + np(m + 1) \text{ (corpo)} \\
&= n + 1 + np + n + npm + np = 2n + 2np + npm + 1 \\
A_{\text{total}} &= 1 \text{ (externo)} + n \text{ (interno)} + np(m + 2) \text{ (corpo)} \\
&= 1 + n + npm + 2np
\end{aligned}$$

O número total de operações primitivas  $N$  é a soma de todas as categorias:

$$\begin{aligned}
N &= F_{\text{total}} + O_{\text{total}} + C_{\text{total}} + A_{\text{total}} \\
&= (1 + n + np) + (n + np + 3npm) + (2n + 2np + npm + 1) + (1 + n + npm + 2np)
\end{aligned}$$

Agrupando os termos semelhantes:

$$\begin{aligned}
N &= (1 + 1 + 1) + (n + n + 2n + n) + (np + np + 2np + 2np) + (3npm + npm + npm) \\
&= 3 + 5n + 6np + 5npm
\end{aligned}$$

Portanto, a fórmula final do número de operações para a multiplicação de matrizes é:

$$N = 5npm + 6np + 5n + 3$$

## Análise de Complexidade

Para o pior caso, onde as matrizes são quadradas ( $n = p = m$ ), a função de custo torna-se:

$$N = 5n^3 + 6n^2 + 5n + 3$$

Desconsiderando as constantes e os termos de menor ordem, obtemos a complexidade assintótica:

$$O(n^3)$$

Este resultado está de acordo com a literatura, que afirma que a multiplicação de matrizes convencional tem complexidade cúbica. A constante 5 no termo dominante é uma informação valiosa: qualquer otimização que reduza essa constante resultará em um algoritmo mais eficiente, mesmo que mantenha a mesma classe de complexidade.

## Conclusão

A análise da complexidade de algoritmos, seja ela iterativa ou recursiva, depende diretamente da capacidade de contar e somar repetições. A **Indução Completa** fornece uma base lógica rigorosa para demonstrar a validade de fórmulas de somatório, enquanto a **Expansão Telescópica** é uma ferramenta prática e eficiente para resolver recorrências lineares, convertendo-as em problemas de somatório. Complementarmente, o paradigma **F.O.C.A.** (Funções, Operações Aritméticas, Comparações e Atribuições) oferece uma abordagem sistemática e detalhada para a contagem de operações primitivas em trechos de código, permitindo a construção de funções de custo precisas. O domínio desses métodos é, portanto, fundamental para a compreensão e o projeto de algoritmos eficientes.